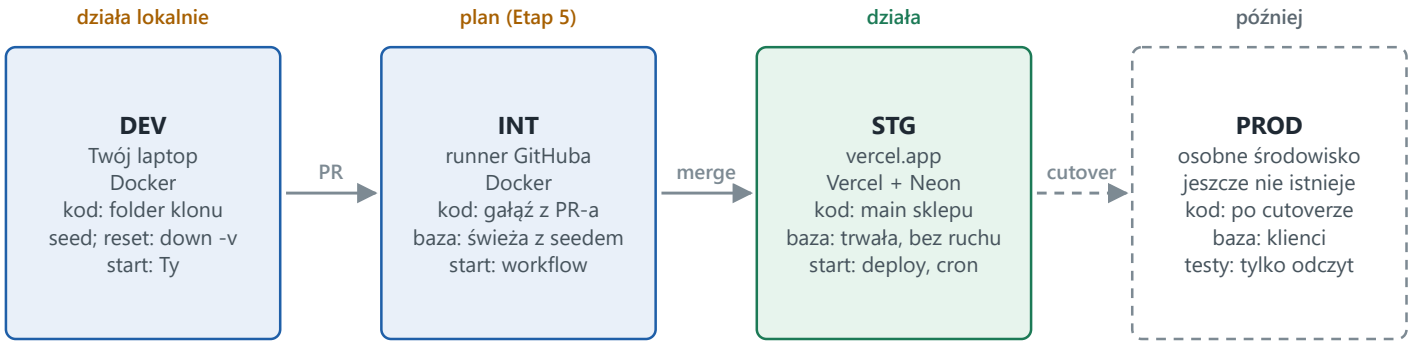
Mapa środowiska testowego

jersa-storefront + jersa_test_framework · stan na 5 listopada 2026

Gdzie działa sklep, gdzie działają testy i co się dzieje od zmiany w kodzie do wdrożenia.

Droga zmiany przez cztery środowiska



Każde środowisko odpowiada na inne pytanie. DEV: czy mój test działa? INT: czy ta zmiana sklepu niczego nie psuje? STG: czy wdrożona wersja działa? PROD: czy sklep działa dla klientów?

Trzy rzeczy do zapamiętania

Docker stawia tylko sklep i bazę

Framework działa poza kontenerem: na Twoim laptopie albo na runnerze. Dzięki temu breakpointy i podgląd przeglądarki działają od razu.

Testy są wszędzie te same

Między środowiskami zmieniają się adresy w `.env`, nie kod testów.

CI to automat, nie bramka

CI uruchamia testy i pokazuje wynik. Bramką staje się dopiero wtedy, gdy reguła ochrony `main` blokuje merge przy porażce.

Dlaczego nie ma stałego serwera INT

Sklep to jedna aplikacja, a jedyna zależność zewnętrzna (PayU) jest na mocku. Tymczasowy Docker na runnerze wystarczy: powstaje na czas testów, ma czystą bazę i nie ma do niego kolejki. Wspólne INT wróciłoby przy wielu serwisach albo prawdziwych integracjach (Etap 8).

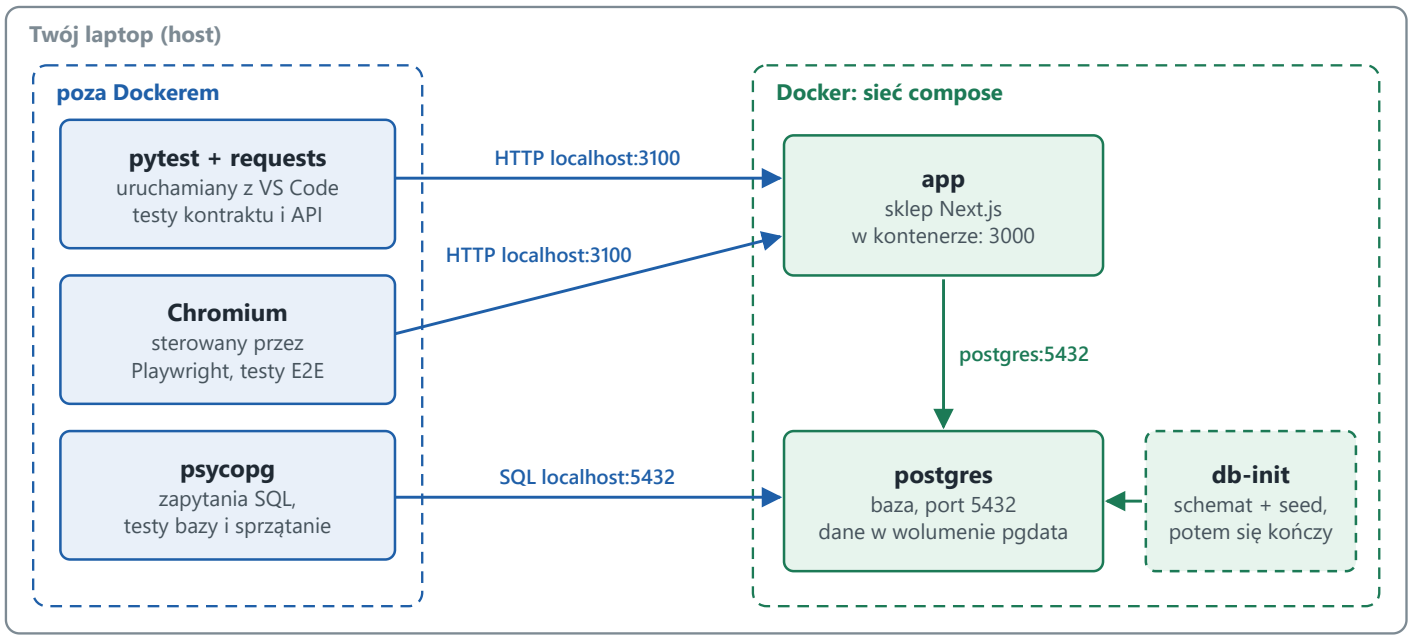
Kto za co odpowiada

Kto	Co
Repo sklepu (agent Grok)	kod sklepu, `Dockerfile`, `.dockerignore`, w Etapie 5 workflow bramki na PR
Repo testów (Claude + Ty)	testy, schematy, `docker-compose.yml`, `.env.example`, w Etapie 5 workflow testów i nightly
Ty	decyzje, push, przekazywanie notek z `docs/handoff/` do Groka
GitHub	runner, status przy PR-ze, reguła ochrony `main`, raporty w zakładce Actions

framework testowy · sklep (SUT) · konfiguracja lub rzecz w toku · blokada

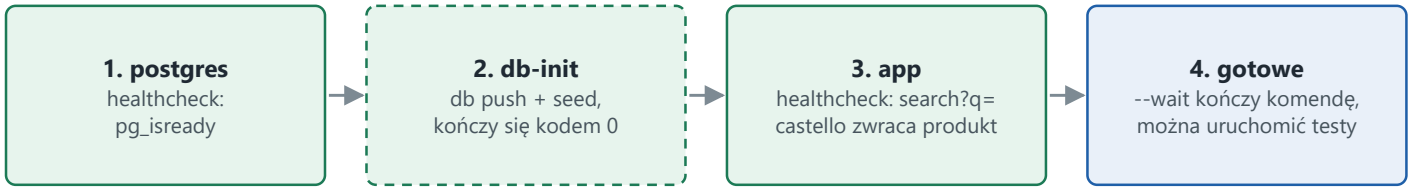
DEV: jak testy widzą sklep na laptopie

Kontenery działają w tle, a testy uruchamiasz z VS Code tyle razy, ile chcesz.



Porty łączą dwa światy. Z laptopa baza to `localhost:5432`, a z kontenera sklepu `postgres:5432`, bo wewnątrz sieci compose usługi widzą się po nazwie. Sklep jest na 3100, bo 3000 zajmuje `next dev` Groka.

Kolejność startu



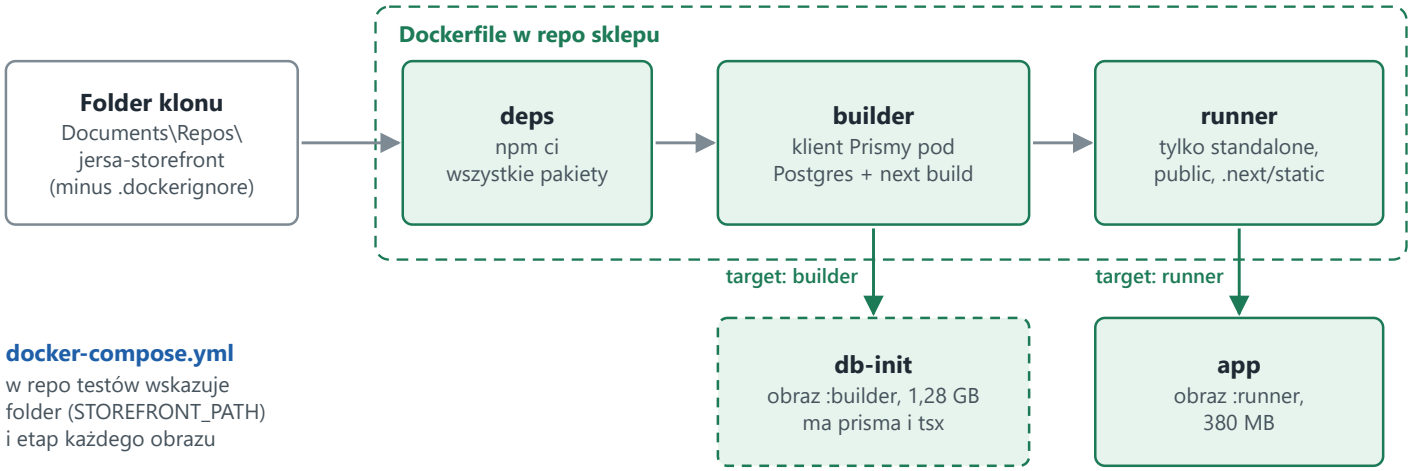
Każda usługa czeka na poprzednią. Jeśli `db-init` się wywróci, sklep nie wystartuje, a przyczynę pokaże `docker compose logs db-init`.

Twój dzień pracy

Kiedy	Co robisz	Co się dzieje
Rano, raz	`docker compose up -d --build --wait`	obraz z aktualnego folderu sklepu, 3 usługi w tle
Cały dzień	▶ w panelu Testing albo `pytest`	testy na hoście, kontenery bez restartu
Grok zmienił sklep	`docker compose up -d --build --wait`	Docker przebudowuje tylko zmienione warstwy
Chcesz czystą bazę	`docker compose down -v`, potem `up -d --wait`	wolumen znika, seed od nowa
Test na STG	`BASE_URL`, przy SQL też `DATABASE_URL`	te same testy; bez SQL Docker niepotrzebny

Skąd bierze się obraz sklepu

Dockerfile leży przy kodzie sklepu, a compose przy testach. Repo aplikacji produkuje obraz, repo testów go używa (ADR-006).



Dwa z trzech etapów Dockerfile stają się obrazami. Duży `builder` tylko przygotowuje bazę, a mały `runner` to sklep, który testujesz.

Co dokładnie trafia do obrazu

Compose **nie pobiera niczego z GitHuba**. Buduje z folderu, w stanie z chwili uruchomienia komendy.

Sytuacja	Co jest w obrazie
Grok zmienił plik, ale bez commita	**jest**, bo pracuje w tym samym folderze
Zmiana jest tylko na GitHubie	**nie ma**, dopóki nie zrobisz `git pull`
`docker compose up` bez `--build`	poprzedni obraz, bez nowych zmian
Runner w CI	konkretny commit z czystego checkoutu

> **Pułapka:** gdy Grok jest w połowie pracy, lokalny obraz zawiera niedokończony kod. Przy niezrozumiałej porażce testu najpierw sprawdź `git status` w klonie sklepu.

Dlaczego przygotowanie bazy jest osobną usługą

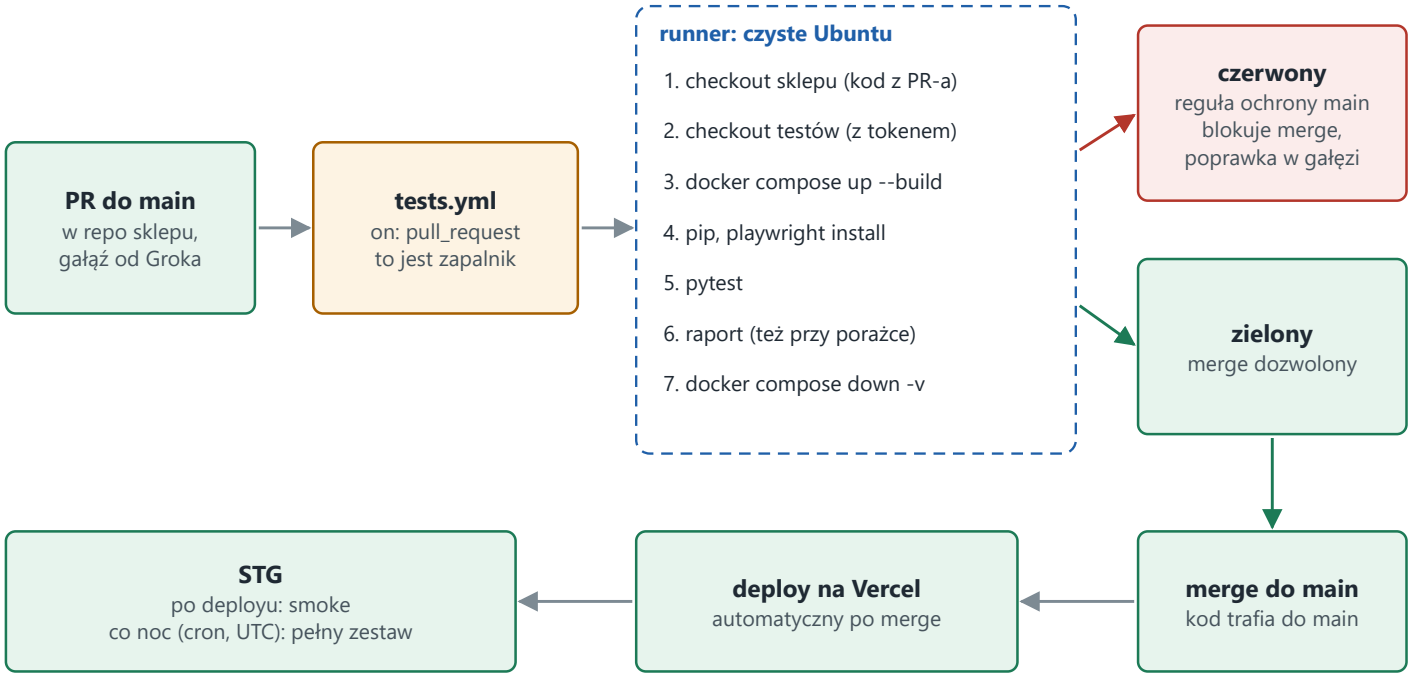
Schemat i seed wymagają Prisma CLI i `tsx`. Oba są w `devDependencies`, więc nie ma ich w obrazie standalone. Zamiast powiększać obraz sklepu, `db-init` używa etapu `builder` i kończy pracę, zanim sklep wystartuje. Dzięki temu aplikacja zawiera tylko to, czego potrzebuje w działaniu.

Miejsce na dysku

Docker trzyma wszystko w pliku `ext4.vhdx`, który sam się nie kurczy. Przed buildem sprawdź wolne miejsce (potrzeba ok. 8–10 GB) i `docker system df`. Pełny dysk zawiesił już Dockera w tym etapie (`KNOWLEDGE.md`, sekcja 6).

INT: bramka na PR w GitHub Actions

Te same kroki co na laptopie, tylko automatycznie i na czystej maszynie. Plan na Etap 5.



Zapalnikiem jest zdarzenie opisane w `tests.yml`, a `docker compose` to tylko krok 3. Maszyna znika po przebiegu, zostają status przy PR-ze i raporty.

Cztery wyzwalacze (ADR-005)

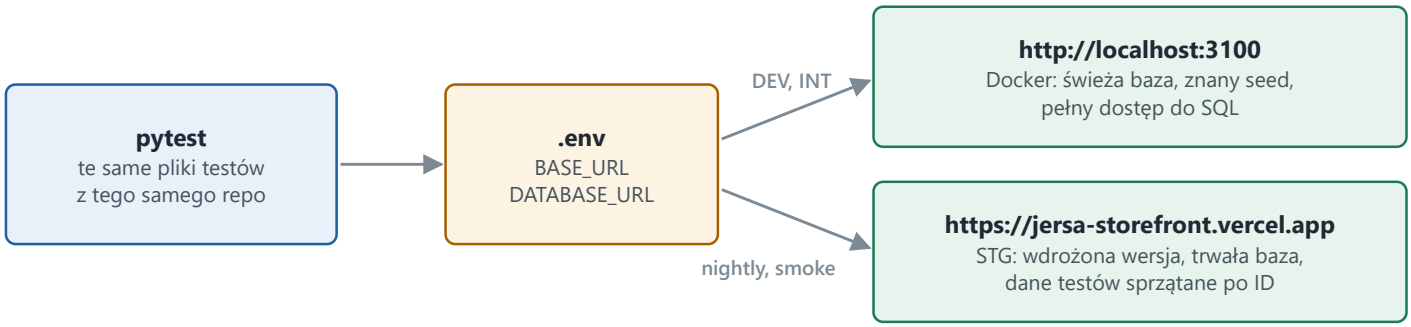
Zdarzenie	Wersja sklepu	Gdzie	Po co
PR w repo sklepu	gałąź PR-a	Docker	główna bramka: błąd nie dociera do `main`
Push lub PR w repo testów	`main`	Docker	czy zmienione testy działają
Co noc (cron)	wdrożona	STG	pełna regresja
Po deployu	wdrożona	STG	szybki smoke

Co trzeba ustawić w GitHubie

1. Wypchnąć repo testów na GitHuba jako prywatne.
2. Utworzyć token z odczytem repo testów i zapisać go w sekretach repo sklepu.
3. Dodać `.github/workflows/tests.yml` w repo sklepu.
4. Włączyć regułę ochrony `main` z wymaganym statusem `tests`.
5. Przejść w sklepie na gałęzie i PR-y.

> **Warunek:** dziś zmiany idą pushem prosto na `main` i od razu na Vercel, więc omijają bramkę. Bez punktów 4 i 5 CI będzie tylko ostrzeżeniem.

Te same testy, różne cele i izolacja danych



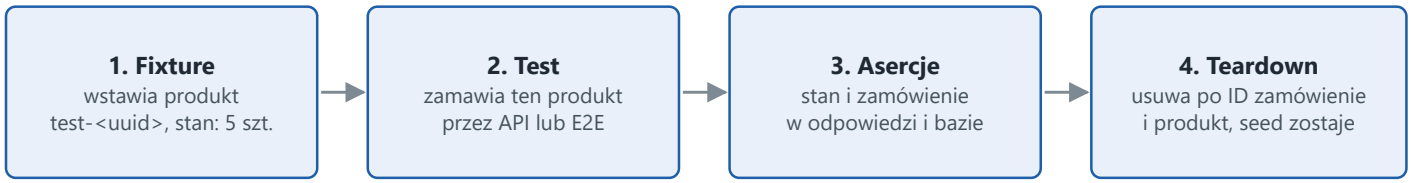
Zmiana celu to zmiana konfiguracji. Docker daje pełną kontrolę nad danymi, a STG sprawdza wersję wdrożoną naprawdę.

Co działa gdzie

Poziom	Biblioteka	Co sprawdza	Docker	STG
Kontrakt	`jsonschema`	kształt odpowiedzi API	**tak**	**tak**
API	`pytest` + `requests`	scenariusze na BFF	**tak**	**tak**
Baza	`psycopg`	stan po operacji	**tak**	**decyzja: sekret Neona w CI**
E2E	Playwright	ścieżki klienta w przeglądarce	**tak**	**tak**

Wersja `psycopg` (2 lub 3) i narzędzie do raportów zostają do rozstrzygnięcia w Etapie 2. Sprzątanie na STG też wymaga SQL, bo API nie ma endpointów kasujących, więc decyzja o sekrecie Neona dotyczy również testów API i E2E, które zapisują dane.

Izolacja na poziomie testu (ADR-003)



Test sprząta tylko po sobie. To propozycja dla zamówień, do potwierdzenia na działającym sklepie w Etapie 3, bo stan magazynowy zapisany jest w `ProductRow.stockBySize`.

	Reset bazy przed zestawem	Własne dane każdego testu
Dwa przebiegi naraz na STG	**jeden kasuje dane drugiego**	**nie przeszkadzają sobie**
Równoległe workery xdist	**wspólna baza bez izolacji**	**każdy ma swoje rekordy**
Na produkcji	**niemożliwy**	**ta sama zasada, testy tylko czytają**
Koszt	mały	każda fixture musi po sobie posprzątać

W Dockerze świeża baza i tak przychodzi za darmo przez `down -v`. Na STG każde zamówienie zmniejszałoby stan seedu, dlatego testy nie kupują seedowych produktów. Seed ma jeden wpis bloga do odczytu; test, który zmienia blog, tworzy własny wpis w ten sam sposób.

Gdzie jesteśmy teraz

Etap 1: sklep w Dockerze	Stan
Projekt, ADR-006 i notka dla Groka	**gotowe**
Dockerfile, `.dockerignore`, `standalone`, `--generate-only` (commity lokalne `4c1a64d`, `4021449`)	**gotowe**
`docker-compose.yml` i `.env.example`	**działają**
Build obrazu w Dockerze	**przechodzi, także z samego commita**
Kryteria akceptacji lokalnie	**spełnione na czystej kopii commita**
Push sklepu i deploy na Vercelu (`91b5669`)	**wdrożone, STG działa**

> **Rozwiązana blokada.** Obraz budowany jest bez dostępu do bazy, a strona `/blog` czyta Prismę bezpośrednio i jest budowana z góry, więc `next build` kończył się błędem `PrismaClientInitializationError`. Strona `/szycie-na-miare` zostawałaby na stałe z danymi z mocka. Po zmianie obie czytają Postgresa, co potwierdził znacznik wstawiony w SQL.

Opcja	Naprawa	Skutek
A	`try/catch` w `/blog`	odrzucona: awaria bazy nadpisałaby cache pustą listą
B	**`force-dynamic` na `/blog` i `/szycie-na-miare`**	**wybrana (ADR-007):** obraz bez bazy, testy widzą Postgresa; koszt: odczyt bazy przy każdym wejściu
C	baza dostępna w czasie `docker build`	odrzucona: build w dwóch fazach
D	tryb `--experimental-build-mode compile`	odrzucona: eksperymentalny

Etap 1 zamknięty (2026-11-06). Dane na STG odbiegają od seedu, bo zmiany z panelu admina zostają. Dlatego testy na STG sprawdzają kształt odpowiedzi i własne dane `test-`, a nie wartości z seedu.

Co dalej

1. **Etap 2:** szkielet frameworka, `conftest.py`, konfiguracja z `.env`, markery, ruff.
2. **Etapy 3–4:** pierwsze testy na czterech poziomach, potem dane i równoległość.
3. **Etap 5:** GitHub Actions, praca przez PR-y, bramka.

Słownik

SUT	system testowany, czyli sklep; framework to narzędzie, nie SUT
Seed	znane dane startowe: 5 produktów, po 5 szt. na rozmiar
Healthcheck	polecenie sprawdzające, czy usługa jest naprawdę gotowa, a nie tylko uruchomiona
Build context	folder, z którego Docker buduje obraz
Workflow / trigger	plik YAML opisujący kroki / zdarzenie, które go uruchamia
Runner	tymczasowa maszyna GitHuba, kasowana po przebiegu
Bramka	punkt, przez który kod nie przejdzie bez spełnionego warunku